

DRAWBACKS OF FILE SYSTEM

1. Data Redundancy & Data inconsistency

- hard to manually check for duplicate data.
- if changes are not made in each place, it becomes inconsistent.
- redundant data, hence consistency issues.

2. Difficulty in accessing data

3. Data isolation

- not easily accessible to all users.
- need to know location of the file.

4. Integrity Problems.

- constraints on data
- type of data that needs to be stored.

5. Atomicity Problems

- should be fully executed or returned back, all or nothing.

6. Concurrent Access Anomalies

7. Security

- should be secured from unauthorised access.
- cannot set roles for different users.

PURPOSE OF DBMS

1. Fast Data Access

- response time increases.

2. Minimized data redundancy → same data can't be stored in more than one place.

3. Data consistency

4. No attributes for accessing data. → no need to know file location.

5. Concurrent Access → multiple users can access.

6. Security

ACID PROPERTY

1. Atomicity → should be performed entirely or not at all.

2. Consistency.

3. Isolation → execution of transaction should not be interfered with other transac.

4. Durability → changes are persisted permanently, should not be lost because of failure.

★ Pakistan foundation.

has large data on employees, students

• concurrent access (performance)

• fast (response time)

• changes made (consistency)

• access restricted (security)

• survive system

• crashes (durability)

TYPES OF DATA

1. Structured Data

- ↳ screenshots, star ratings.
- ↳ tabular format.
- ↳ clearly defined
- ↳ name, address.
- ↳ relational DB

2. Unstructured Data.

- ↳ no data model
- ↳ no pre defined.
- ↳ irregular and ambiguous.
- ↳ non-relational
- ↳ no SQL
- ↳ text, video, audio

3. Semi-Structured Data.

- ↳ emails, HTML

TYPES OF DATABASE

• SQL DB are structured DBMS

• noSQL is → next-generation DB, big data, horizontally scalable.

• graph, document, key-value and column database.

ARCHITECTURES



R



C

Key-value

Key-value

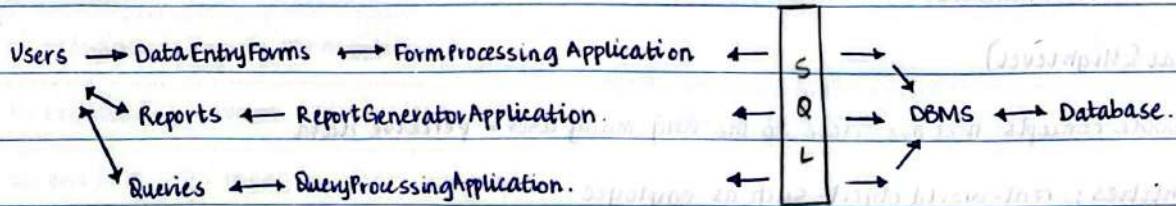
SQL

- pre defined schema
- add Nil if empty data → memory wastage
- not easily scalable
- requires joins.
- change schema in case of modification

NOSQL

- schema-less database.
- changes can be easily incorporated.

ARCHITECTURE



DATA ABSTRACTION

• also known as three schema architecture.

1. Physical/Internal level

↳ how data is actually stored.

↳ physical storage structure of the database.

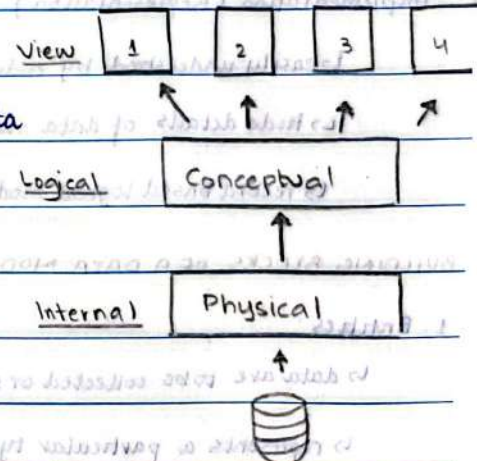
↳ complete details of data storage and access path for database.

2. Conceptual/Logical level

↳ describes structure of database for community of users, what data

↳ hides the details of physical storage structures

↳ entities, datatype, relationships, user operations, constraints.



3. External/View level

↳ describes part of database that a user group is interested

↳ hides rest of the database from that user group.

DATA MODELS

- set of concepts to describe the structure of a database
- structure means data types, relationships, constraints that apply to the data.
- operations for manipulating these structures, retrievals and updates on database
- constraints specify restrictions on valid data.
- operations include insert, delete, update, and user defined operations.

CATEGORIES OF DATA MODELS.

1. Conceptual (High level)

- ↳ provide concepts that are close to the way many users perceive data.
- ↳ entities: real-world object such as employee
- ↳ attribute: property of interest that further describes entity, such as employee name.
- ↳ relationship: two or more entities represents association

2. Physical (Low level)

- ↳ describes how data is stored in the computer
- ↳ generally meant for computer specialists, not for end users.

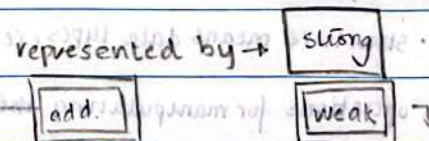
3. Implementation (Representation)

- ↳ easily understood by end users
- ↳ hide details of data storage on disk, but can be implemented on a computer system.
- ↳ record based logical models: relational, hierarchical, network (graph) (each record have more than 1 PR)

BUILDING BLOCKS OF A DATA MODEL

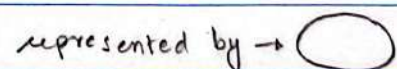
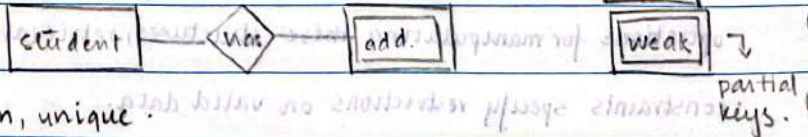
1. Entities

- ↳ data are to be collected or stored. → person, place, thing. is a singular noun, specific
- ↳ represents a particular type of object in the real world, are "distinguishable"
- ↳ occurrence is unique and distinct → customer (tabidah, umema, manyam, noor)
- ↳ tangible entity: physical occurrence → customer, product, shop.
- ↳ intangible entity: abstractions like product buying.



2. Attributes

- ↳ characteristic of entity. is a noun, unique.
- ↳ customer entity (fname, lname, number, address, etc)
- ↳ equivalent of field in file system.



3. Relationship

- ↳ association among other entities.
- ↳ relationship between customer and agents → agent can serve many customer
customer is served by one agent.
- ↳ one to many, many to many, one to one



4. Constraints

- ↳ restriction placed on data
- ↳ ensure data integrity → accuracy, completeness
- ↳ constraints are normally expressed in form of rules

DATABASE DESIGN

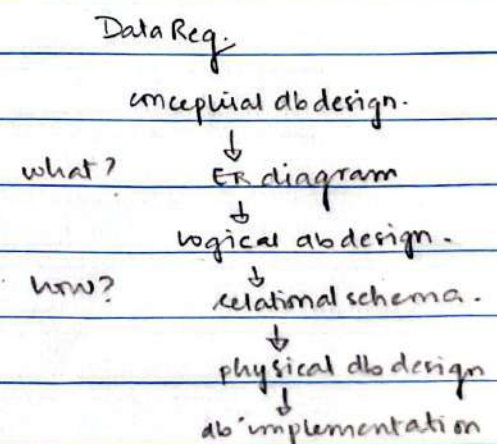
* a good database design is

- ↳ divides information into subject-based tables.
- ↳ requires access to information that joins tables.

DESIGN PROCESS

- purpose of database
- organize information required.
- table
- information items into columns.
- specify primary keys
- table relationships
- refine your design.
- apply normalization rules.

→ not every table has a primary key.



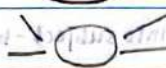
ENTITY RELATION MODEL

- shows conceptual schema, high level.
- overall logical structure
- data modelling technique.
- transformed into logical model.
- widely used in database design.
- entities, attributes, relationships.
- simple, effective communication tool, easy conversion to any model.

1. simple attribute → atomic, cannot be divided



2. composite → made up of more than one single attribute.



3. single valued → takes only one value at a time

4. multi-valued → takes more than one value at a time.



mobile number
email id.

5. derived → values can be derived from other attributes.



dob
age

6. key → uniquely identified in an entity set.



7. descriptive → additional information about relationship.

RELATIONSHIPS

- association between instances of one or more entity types.
- given a name that describes its function

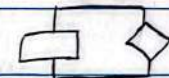
→ active or passive verb

Author ↔ Book



- number of entity type that participates in a relationship.

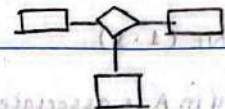
- unary → b/w two instances of one entity type



- Binary → two entity sets participation in a relationship. (degree 2)

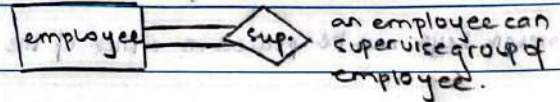


- Ternary → three entity sets participation in a relationship. (degree 3)



UNARY RELATIONSHIP

- recursive relationship
- between different instances of same entity type.

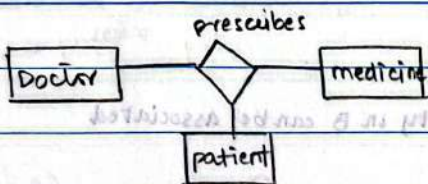


BINARY RELATIONSHIP



enrolls is a relationship between student and courses.

TERNARY RELATIONSHIP

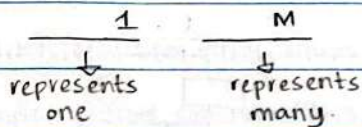


patient goes to a doctor, and doctor prescribes medicine to the patient.

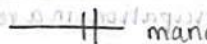
MAPPING CARDINALITIES / CONSTRAINTS

- mapping constraints defines how many entities can be related to another entity
- useful in describing relationship set.
- Cardinality: minimum and maximum number of instances of Entity B that can or must be associated with each instance of entity A.

Chen Model



Crow's Foot



ONE TO ONE (1:1)

- an entity in A is associated with at most one entity in B and vice versa.
- the two entities could belong in the same table.
- foreign keys can be placed in either of the entities.



ONE TO MANY (1:M)

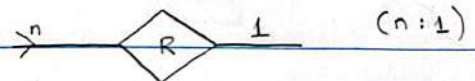
- an entity in A is associated with any (zero or more) entities of B, and an entity in B can be associated with at most one entity in A.



- relational modelling ideal

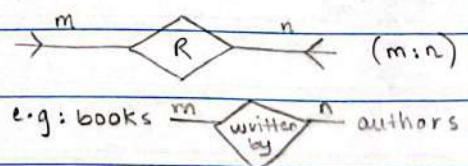
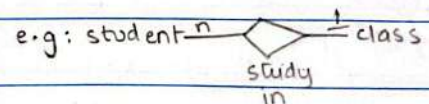
MANY TO ONE (M:1)

- an entity in A is associated with at most one entity in B, and an entity in B can be associated with any (zero or more) number of entities in A.



MANY TO MANY (M:N)

- an entity in A is associated with any number of entities in B and an entity in B is associated with any number of entities in A.
- must be avoided because they lead to data redundancy.
↳ create a composite bridge or bridge entity.



CHARACTERISTIC OF TABLE

- must have distinct or unique name
- no two attributes should have same name.
- no duplicate tuples.

INTEGRITY CONSTRAINTS

- conditions applied on data to check consistency, and correctness.

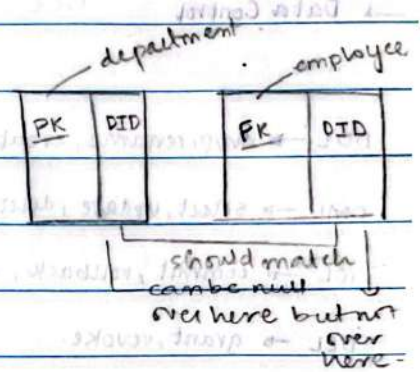
Integrity Constraints

1. Domain
2. Entity
3. Referential
4. Key.

- ↓
- should be in its domain of attribute
 - example: value '2' is not allowed since positive integers can be taken by age.

- ↓
- primary key should not be null nor duplicate as it is uniquely identified.
 - example: roll-no of a student

- ↓
- between two tables; foreign key value must match primary key value.
 - or foreign key should be null.
 - can't delete a record from primary table if matching records exist in a child table.
 - can't change a primary key in the primary table if child record exists.



SQL

- domain specific
- not case sensitive

→ Data Definition DDL } statements for data definitions, queries and updates.
→ Data Manipulation DML
→ Transaction Control TCL
→ Data Control DCL

DDL → drop, rename, create, alter, truncate

DML → select, update, delete, insert.

TCL → commit, rollback, savepoint.

DCL → grant, revoke.

DATA TYPES

- a table with inappropriate data types lead to issues;

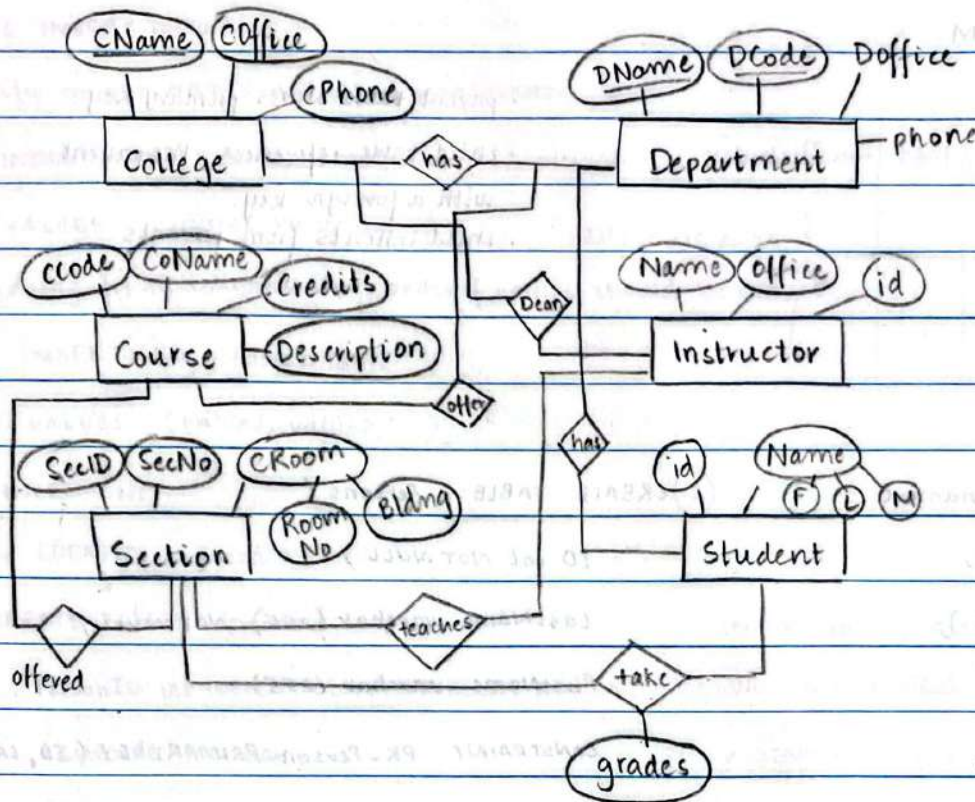
↳ improper query optimization

↳ performance issues

↳ data truncation.

- numeric → integer, int, smallint, float, real, double precision.
 ↳ fixed length ↳ variable-length.
- character → char, character, varchar, char varying, character varying.
- date → YYYY-MM-DD

SPRING 2023

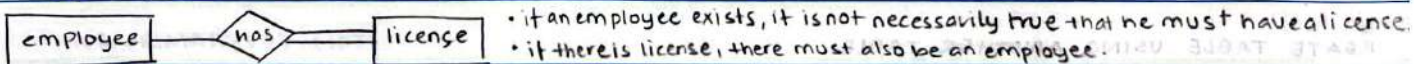


- * there cannot be two primary key
- * ATM structured
- * facebook unstruc
- * customer reviews semi-structured
- * email semi-structured
- * phonebook structured

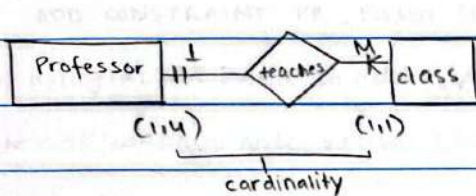
PARTICIPATION CONSTRAINTS

Total participation → if every entity in E participates in atleast one relationship in R * indicated by double line

Partial participation → if only some entity in E participate in relationships in R * indicated by single line.



(Min, Max) notation to tell minimum and maximum participation.



a professor can teach many class, but a class can be taught by a professor.

RELATIONAL MODEL DIAGRAM

EmpID	Ename	Salary	Add

PK → attributes

} rows are tuples

- parent table stores primary key.
- child table references the parent with a foreign key.
- child inherits from parents.

SQL CREATE TABLE

CREATE TABLE table_name (

column1 datatype,

column2 datatype);

① CREATE TABLE Persons (

ID int NOT NULL,

LastName varchar (255) NOT NULL,

FirstName varchar (255)

CONSTRAINT PK-Person PRIMARYKEY (ID, LASTNAME)

);

• not null enforces a column to not accept null values.

• primary key must contain unique values and not null.

• above table has primary key of 2 columns.

CREATE TABLE USING ANOTHER TABLE

- a copy can also be created.

CREATE TABLE TestTable AS

SELECT customername, contactname

FROM customers;

2. SQL INSERT INTO

- specify column names and values to be inserted.

INSERT INTO table-name (column1, column2, ...)

VALUES (value1, value2, ...);

- if adding for all columns then order of values should be correct

INSERT INTO table-name

VALUES (value1, value2, ...);

AUTO INCREMENT

- uses IDENTITY keyword

CREATE TABLE Persons (

PersonID int IDENTITY(1,1) PRIMARY KEY,

Lastname varchar,

Firstname varchar);

INSERT INTO Persons (Firstname, Lastname)

VALUES ('Tabidah', 'Usmani');

- * IDENTITY (1,1) means starting value is 1 and for each record it will increment 1.

3. ALTER TABLE SQL

- to create a primary key constraint on the ID column when table is already created

ALTER TABLE Persons

ADD PRIMARY KEY (ID);

- to allow naming of primary key, and for defining PK on multiple columns

ALTER TABLE Persons

naming

multiple columns.

ADD CONSTRAINT PK_Person PRIMARY KEY (ID, lastname);

- * if using ALTER TABLE to add a primary key, the columns must have been declared to not contain NULL values (when table was created)

- * drop column, add column, drop default, set default, drop constraint

- SQL foreign key → table with foreign key is child table

table with primary key is parent table.

CREATE TABLE Persons (

ID int NOT NULL,

Firstname varchar,

Lastname varchar,

PRIMARY KEY (ID));

CREATE TABLE Orders (

OrderID int NOT NULL PRIMARY KEY

OrderNumber int NOT NULL

PersonID int FOREIGN KEY REFERENCES Persons (ID)

);

parent table
PK

• to create a foreign key constraint on 'PersonID' when 'Orders' table is already created.

ALTER TABLE Orders

ADD FOREIGN KEY (PersonID) REFERENCES Persons (ID)

4. SQL DELETE

rows deletion

DELETE FROM table_name WHERE condition;

• use DROP TABLE to remove table.

5. SQL SELECT

SELECT column1, column2, ...

FROM table_name;

• data returned is stored in result_table called result-set

SELECT column1, column2, ...

FROM table_name

WHERE tuples

- AND operator displays record if all conditions are true.
- OR operator displays record even if one condition is true.
- NOT operator displays record if conditions not true.
- LIKE used to search for a specified pattern in a column.

'a%' → values starting with 'a'

'%a' → values ending with 'a'

'%o or %' → values having 'or' in any position

'_r%' → values having 'r' in second place

'a-%' → values start with 'a' are having 2 characters atleast

'a_-%' → " " " " 3 " "

'a%o' → values start with a and ends with o.

- ORDER BY sorts record in ascending by default

6. SQL UPDATE

UPDATE table_name

★ omitting where clause will

SET column1 = value1, column2 = value2, ...

update all records in table.

WHERE condition ;

EXTENDED FEATURE OF ER MODEL

• enhancements are

↳ subclasses, superclasses, inheritance.

↳ specialization, generalization

↳ shared subclass (multiple inheritance)

• sometimes entities can be classified to specific entities which have its own characteristics.

Employee → secretary, engineer, manager, technician

∴ here employee is a superclass, and others are subclass.

↳ is-a relationship, represented by

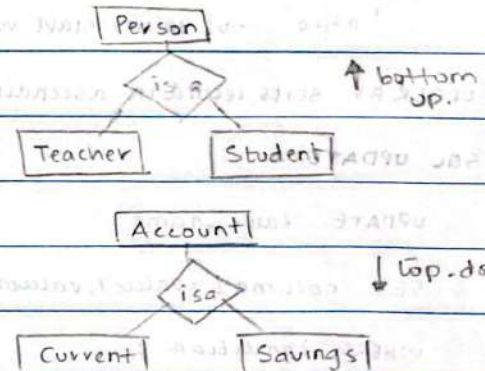
• generalization is a bottom up design - two or more lower level entities are combined.

↳ subclass combined to make a super-class.

∴ arrow pointing to generalized super-class.

• specialization is a top down approach

∴ defining set of sub-classes.



CONSTRAINTS

1. disjointness

1. total specialization

2. overlapping

2. partial specialization

1. subclasses must be disjoint

↳ if an entity belongs to one subclass of a specialization, then it cannot belong to another subclass of same specialization.

2. subclasses can overlap

↳ entity can belong to more than one subclass of same specialization.

• total specialization

↳ every entity that belongs to a superclass must belong to atleast one subclass.

↳ represented using double lines.

• partial specialization

↳ some entities might not belong to any of the subclass of specialization

one - many, many - many, one - one, many - one.

E-R MODEL

• graphical drawing of entities and attributes

• relationships and not tables

RELATIONAL MODEL

• the database is viewed as a table and their attributes (keys)

• not all attributes are attributes on an entity, some might be establishing relationships.

E-R DIAGRAM TO RELATIONS

• entity type becomes a table

• all single valued attribute becomes a column

• key attribute of entity type represented by primary key.

• composite attribute represented by components.

• multivalued attribute represented by a separate table.

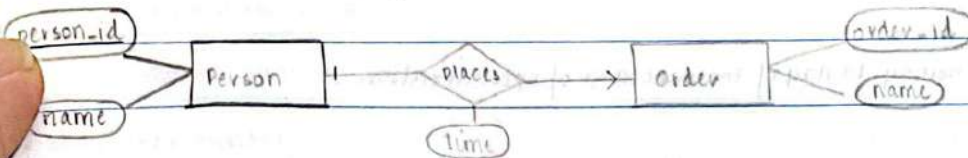
• derived attributes are not considered in the table.

• simple attributes of composite will be represented in the table e.g F.name, L.name and DOB, name.
• if in same table then redundancy of other attributes, key cannot be the same, (1, 1) primary
• coded separately in SQL as view.

RELATIONSHIPS

1. MAPPING OF 1:MANY

the primary key attributes of the "1" relation becomes foreign key attribute of "many" relation.

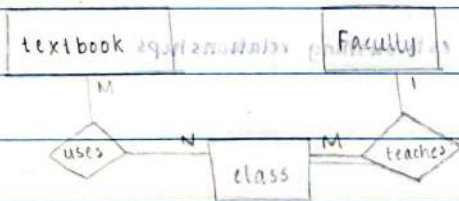


Person (Person-id, name)

Order (Order-id, name, time, Person-id)

2. MAPPING OF MANY:MANY

relationships are always mapped to a separate relation.



Textbooks (ISBN, Title, Author, Copyright, Edition, Price)

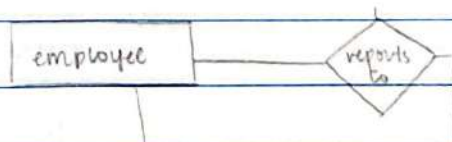
Class (ClassNo, Name, Room)

TextUses (ISBN, ClassNo) * includes PK of both table

Faculty (F-ID, name)

3. MAPPING OF 1:1

- a second attribute of the same domain as the key may be added.
- attributes of relation can also be added to entity relation.



Employee (ID, Name, Salary, Supervisor-ID)

MAPPING EER DIAGRAM TO RELATIONS

MAPPING OF SPECIALIZATION OR GENERALIZATION

Option 1: multiple relations superclass and subclass.

option 2: multiple relations - subclass relations only.

option 3: single relation with one type attribute.

Option 4: single relation with multiple type attribute.

OPTION 1

• a class C is specialized into $\{S_1, S_2, \dots, S_m\}$ subclasses.

*superclass and subclass

• attributes of C are $\{K, a_1, a_2, \dots\}$

→ create relation of superclass C and put all attributes

→ create relation of each subclass and put all attributes corresponding to it.

→ add primary key of C that is K in each subclass.

• works for any kind of specialization → Total, disjoint.

OPTION 2

→ create relation for each subclass and put all attributes corresponding to it.

*subclass only

→ put all attributes of superclass C in each subclass.

• works for total specialization.

OPTION 3

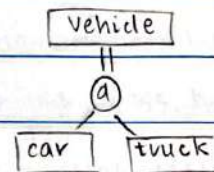
→ create only one relation. put all attributes of superclass C in the relation

*one table with distinguishable attribute.

→ put all attributes of each subclass in the same new relation.

→ add a new attribute to the relation, this attribute used to discriminate between different entity type.

• works for disjoint specialization, values can be null.



OPTION 4

- create only one relation . put all attributes of superclass C in the relation
- put all attributes of each subclass in same new relation.
- add new attributes to the relation , each attribute corresponds to one subclass type
each attribute is a boolean attribute used to discriminate.
- works for overlapped specialization , as well as disjoint

customer (customer-id , customer-name , customer-street , customer-city)

loan (loan-number , amount)

employee (employee-id , employee-name , telephone-number , start-date)

account (acc-number , balance)

payment (payment-number , payment-date , payment-amount , loan-number)

branch (branch-name , branch-city , assets)

dependent-name (dependent-id , employee-id)

account-branch (branch-name , acc-number)

loan-branch (branch-name , loan-number)

borrower (cust-id , loan-number)

depositor (access-date , cust-id , acc-num)

cust-banker (type , cust-id , employee-id)

RELATIONAL ALGEBRA

- uses collection of 'OPERATORS' to compose the queries.
- describes a step by step procedure; mathematical notations are used
- two groups of relational algebra operations;
 - relational database $\rightarrow$ SELECT, PROJECT, JOIN
 - mathematical set theory $\rightarrow$ UNION, SET, DIFFERENCE, INTERSECTION,

CARTESIAN PRODUCT, DIVISION

UNARY RELATIONAL OPERATION

- applied to single relation only.

1. SELECT (σ)

$\rightarrow$ choose a subset of tuples from a relation that satisfies a selection condition.

$$\sigma_{\langle \text{selection condition} \rangle} (R) \quad \sigma_{p(r)} \quad \text{where } p = \text{boolean condition (AND, OR, NOT)}$$

$p = \langle \text{attribute name} \rangle \langle \text{comparison operator} \rangle \langle \text{constant value} \rangle$ or $\langle \text{attribute name} \rangle$

example 1: select all the student of Team A

$$\sigma_{\text{Team} = 'A'} (\text{STUDENT})$$

select * from STUDENT where Team = 'A';

example 2: select all students whose fees is greater than equal to 10000 and belongs to Team other than A.

$$\sigma_{\text{Fees} \geq 10000} (\sigma_{\text{Team} \neq 'A'} (\text{STUDENT}))$$

$\rightarrow$ select operator is applied to each tuple individually.

$$\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (R)) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle} \dots \quad (\text{order doesn't matter, is commulative})$$

SQL

Parser

Relational Algebra Expression

Query optimize

Query Execution Plan

Executable Code

Code Generator

2. PROJECT (π)

→ selects certain columns from table and discards other.

* also known as vertical partition

→ duplicate rows are automatically eliminated, as relation is a set.

$$\pi_A(R) \quad \pi_{\langle \text{attribute list} \rangle}(R)$$

example 1: display class and dept from faculty

$$\pi_{\text{class, dept}}(\text{Faculty})$$

select distinct class, dept FROM Faculty.

$$\rightarrow \pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$$

SELECT and PROJECT EXAMPLE

- select name of student of Team A

$$\pi_{\text{name}}(\sigma_{\text{Team}='A'}(\text{student}))$$

3. RENAME (ρ)

→ renames output of a relation.

→ save result of relational algebra expression as a relation.

→ or to join a relation with itself.

$$\rho_{\text{newname}}(\text{tablename})$$

example 1: rename student table

$$\rho_{\text{newstudent}}(\text{student})$$

example 2: section, branch column rename

$$\rho_{\text{newsection, newbranch}}(\pi_{\text{section, branch}}(\text{student}))$$

example 3: rename student relation to malestudent and student, rollno as Sno, Sname

$$\rho_{\text{malestudent}}(\text{Sno, Sname})(\pi_{\text{rollno}}(\text{studentname}(\text{student})))$$

breaking down complex sequence: $\text{TEMP} \leftarrow \sigma_{D.NO=3}(\text{EMPLOYEE})$

$$\text{RESULT} \leftarrow \pi_{(\text{Fname, LName})}(\text{TEMP})$$

also written as: $\pi_{\text{Fname, LName}}(\sigma_{D.NO=3}(\text{EMPLOYEE}))$

ALGEBRA OPERATION FROM SET THEORY

1. CARTESIAN PRODUCT (CROSS PRODUCT)

→ used to merge columns from two relations.

→ denoted by $\times$ relation1 $\times$ relation2

→ columns = $M+N$

rows = $M \times N$

→ use it as $\sigma_A = D (A \times B)$

example 1:

$\sigma_{age \geq 15} (\text{student} \times \text{reg-d})$

2. SET DIFFERENCE

→ result of $(A-B)$ is a relation which includes all tuples that are in A but not in B.

→ attribute name of A and B should match.

→ should be compatible or union compatible.

example 1: find regno of students who plays indoor sports but not outdoor.

$\pi_{\text{regno}} (\text{stu-indoor}) - (\pi_{\text{regno}} (\text{stu-outdoor}))$

select * FROM borrow EXCEPT select * FROM depositor

3. UNION

→ contains all tuples that are either in R or S or both in R and S

→ eliminates duplicate

→ R and S must have attribute of same number

example 1: $\pi_{\text{author}} (\text{Books}) \cup \pi_{\text{author}} (\text{articles})$

Female_amps $\leftarrow \sigma_{\text{sex} = 'F'} (\text{employee})$

emp-names $\leftarrow \pi_{\text{fname}, \text{lname}, \text{ssn}} (\text{female_amps})$

emp-dependents $\leftarrow \text{emp-names} \times \text{dependent}$

actual-dependants $\leftarrow \sigma_{\text{ssn} = \text{Essn}} (\text{emp_dependants})$

RESULT $\leftarrow \pi_{\text{fname}, \text{lname}} (\text{actual_dependants})$

4. INTERSECTION

→ includes all tuples that are in R and S

example 1: $\pi_{\text{customer_name}}(\text{borrow}) \cap \pi_{\text{customer_name}}(\text{depositor})$

BINARY RELATIONAL OPERATION

1. DIVISION

→ evaluate queries which contain keyword ALL, ATALL, EVERY

$$R1 \div R2 \rightarrow A(x, y) / B(y)$$

→ finds records in R1 that is related to all records in R2

→ can only be applied if attributes of B is a proper subset of A

→ output will be attributes = (all in A - all in B)

Student Enrolled in the Course $A(x, y) / B(y)$

S-id	C-id	C	studentenrolled(S-id, c-id) / course(C-id)
S1	C1	C2	
S2	C1		
S1	C2		
S3	C2		

→ can be expressed as

$$T1 \leftarrow \pi_y(R)$$

$$T2 \leftarrow \pi_y((S \times T1) - R)$$

$$T \leftarrow T1 - T2$$

2. JOIN

→ cartesian product followed by selection criteria.

→ denoted by $R_1 \bowtie R_2$

→ allows joining related tuples from different relations.

example 1: display managers of each department.

• Dept-Mgr $\leftarrow$ DEPARTMENT $\bowtie$ Mgr-SSN = SSN EMPLOYEE

• Temp $\leftarrow$ DEPARTMENT $\times$ EMPLOYEE

Dept-Mgr $\leftarrow$ $\sigma_{\text{MgrSSN}=\text{SSN}}$ (Temp)

example 2:

Student $\bowtie$ student.id = student-course.course_id Student Course $\bowtie$ student-course.course_id = course.id Course

→ theta join: join R_1 and R_2 other than equal to condition.

$A \bowtie_{\theta} B \quad R \bowtie_{\text{condition}} S$

→ equi join, uses only equivalence condition.

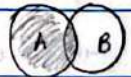
$A \bowtie_{A.\text{column}_2 = B.\text{column}_2} B$

→ natural join: if there is a common attribute (column)

name and type must be same.

$R_1 \bowtie R_2$

→ left outer join: contains set of tuples of all combinations in A and B that are equal on their common attribute name, tuples in A have no matching tuples in B .



→ right outer join: same as left outer join, but tuples in B that have no matching tuples in A .

$\bowtie$

→ full outer join: it contains all rows from both tables.

$\bowtie$

DBMS KEYS

- uniquely identify any record or row of data.
- establish and identify relationships between tables.
- Key's role is based on a concept known as determination.

1. SUPER KEY

- uniquely identify each record
- maximum super keys = $2^n - 1$ (n attributes)

BSN	Name	Marks	Dept	Course
1	Ali	78	CS	101
2	Ahmed	89	SE	102
3	Sultan	75	SE	101
4	Ali	78	CS	103
5	Rida	82	CS	104

2. CANDIDATE KEY

- set of attributes that can uniquely identify
- is a super key whose proper subset is not a superkey. (called minimum superkey)
- can contain null values.

* [Name, marks] cannot be a superkey

natid $\in$ (natid, cname), then (natid, cname) is not candidate key and (natid) is a candidate key.

3. PRIMARY KEY

- identifies only one instance of an entity.
- may be a composite key.
- a primary key is a superkey and candidate key.

4. UNIQUE KEY

- can accept null values.
- can be more than one unique key.

5. FOREIGN KEY

- a column of the table used to point to the primary key of another table.
- a foreign key is null when value in parent table is not defined or value in parent table is deleted.
- foreign key can refer to primary key of same relation.

6. SURROGATE KEY

- artificial column added to serve as a primary key.
- values meaningless to users.

Cascade → if parent record is deleted, child record is also deleted.

noaction → rejects delete or update operation for parent table if there is a related FK value in the referenced table.

DELIMITER //

-- drop procedure getCustomers.

CREATE PROCEDURE getCustomers ()

BEGIN

#query

END //

DELIMITER ;

call getCustomers.

-

-

Roll No	First Name	Last Name	Marks
---------	------------	-----------	-------

- * if we know roll-no, we can tell
firstname, lastname, marks but
not vice versa.

$X \rightarrow Y$ means ; X functionally determines Y

X arrows to Y

Y is associated with X

X is called the determinant , Y is dependent.

example: Movie (title, year, length, filmtype, studioname, starname)

title, year $\rightarrow$ length, filmtype, studioname

TYPES OF FUNCTIONAL DEPENDENCIES.

1. FULLY FUNCTIONAL

if X and Y are attribute of a relation, Y is fully functionally dependent on X ,

but not on subset of X

example : SupplierID, ItemID, Price.

SupplierID, ItemID $\rightarrow$ Price is correct

SupplierID, Price is incorrect // not on subset

ItemID, Price is incorrect // not on subset

2. PARTIALLY FUNCTIONAL

if some attribute can be removed from X , and yet dependency holds.

example : StudentID, DeptID, DeptName

StudentID, DeptID $\rightarrow$ DeptName is correct

DeptID $\rightarrow$ DeptName is correct

StudentID $\rightarrow$ DeptName is incorrect

example : StudentID, StudentName, CourseID, CourseName, Grade.

StudentID, CourseID $\rightarrow$ StudentName, CourseName, Grade — fully functionally.

CourseID $\rightarrow$ StudentName.

StudentID $\rightarrow$ Grade is incorrect

StudentID $\rightarrow$ StudentName is incorrect

3. TRIVIAL FUNCTIONAL

- if Y is a subset of X

example. StudentID, StudentName, Branch, Marks.

StudentID, StudentName, $\rightarrow$ StudentID

StudentID, StudentID.

4. NON-TRIVIAL FUNCTIONAL

- if Y is not a subset of X

5. COMPLETELY NON-TRIVIAL

- if X intersection Y is null.

6. TRANSITIVE FUNCTIONAL

- if X, Y, Z are attributes of R

- if $X \rightarrow Y$ and $Y \rightarrow Z$, then Z is transitively dependent on X .

FD1 : StudentID $\rightarrow$ DeptID.

FD2 : DeptID $\rightarrow$ DeptName

FD3 : StudentID $\rightarrow$ DeptName

FIRST NORMAL FORM (1NF)

if every row contains exactly one value for each attribute.

Dname	Dnumber	Dmgr-ssn	Dlocations
Research	5	333445555	Bellaire, sugarland, Houston



Dname	Dnumber	Dmgr-ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	sugarland
Research	5	333445555	Houston

SECOND NORMAL FORM

- firstly it should be in 1NF
- then every non-key attribute is fully dependent on primary key.

EMP_PROJ

	Ssn	Pnumber	Hours	Ename	Pname	Plocation
FD1			↑			
FD2				↑	↑	↑
FD3						↑

(Ssn, Pnumber) → Hours full

Ssn → Ename partial

Pnumber → Pname, Plocation partial

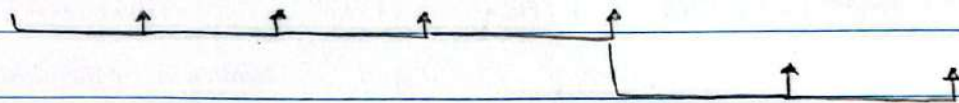
IN
2NF

Ssn	Pnumber	Hours	Ssn	Ename	Pnumber	Pname	Plocation
-----	---------	-------	-----	-------	---------	-------	-----------

THIRD NORMAL FORM (3NF)

- it is in 2NF
- every non-key attribute is non-transitively dependent on primary key.

Ssn Ename Bdate Address Dnumber Dname Dmgr_ssn.



IN
3NF

FIRST TABLE

Ssn Ename Bdate Address Dnumber

SECOND TABLE

Dnumber Dname Dmgr_ssn

4NF: inappropriate usage of a M-way. does not involve functional dependencies.

5NF: concerned with join dependencies.

example: stdSSN stdclass stdcity offerNo offYear offTerm EndGrade CourseNo Crs

stdSSN → stdcity, stdclass

offerNo → offTerm, offYear, CourseNo, CrsDesc.

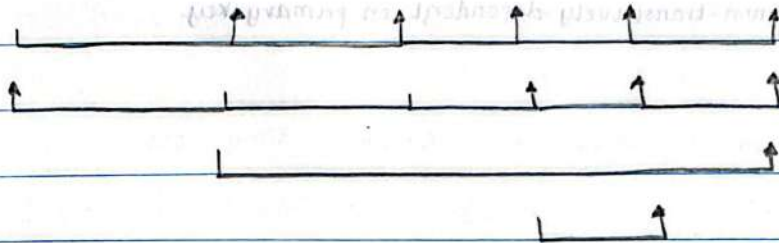
CourseNo → CrsDesc

stdSSN, offerNo → EndGrade.

example:

candidate key.

property-id county-name lot# area price tax-rate



IN 2NF

property-id county-name lot# area price

county-name tax-rate

}

IN 3NF area price property-id county-name lot# area

TRANSACTION OPERATIONS

Read: read value from database, it does not update any value in database.

Write: it writes data into the database. it has to be committed to make updated information available.

operations involved:

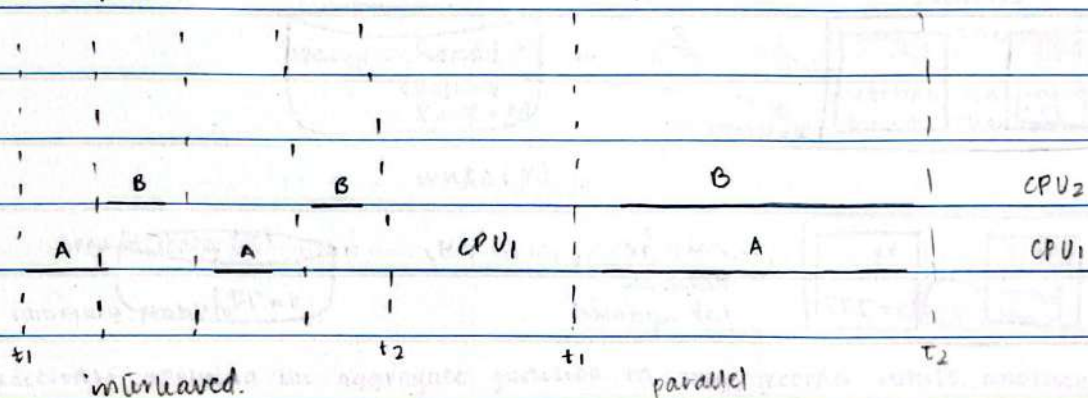
1. read data
2. modify read data values
3. write data back
4. commit transaction.

read-write: performing operations by different users on the same data at the same time.
may arise data conflict.

CONCURRENT TRANSACTION

(*) multi - processing

- A single CPU can only execute at most one process at a time.
- Multiprogramming executes some commands from one process, then suspends the process and execute some command from next process. A process resumes from the point where it was suspended. hence concurrent transaction is interleaved.
- If a computer system has multiple hardware processors, parallel processing of multiple processors is possible.



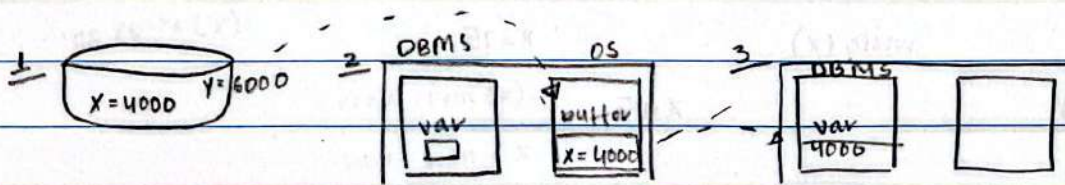
- concurrency relates to an application that is processing more than one task at the same time.

- decreasing response time of system using a single processing unit.

READ AND WRITE OPERATIONS.

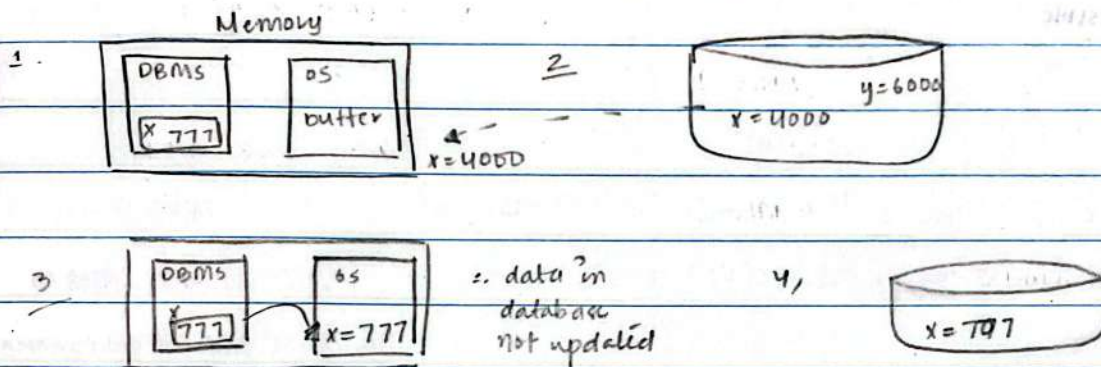
- read-item (x)

1. find the address of disk block that contains item x
2. copy the disk block into a buffer in main memory.
3. copy the item x from the buffer to the program variable named x.



• write-item (x)

1. find address of the disk block that contains item x
2. copy disk block into a buffer in main memory
3. copy item x from the program variable named x into its correct location in the buffer.
4. store updated block from the buffer back to disk



PROBLEMS IN CONCURRENT TRANSACTION

1. lost update problem

- when two transactions are updating same record at the same time in a DBMS

Transaction A	Transaction B	A	B	
Read(x)		X=100		∴ it should have been for transac B
X = X + 15		X=115		
	Read(x)		X=100	X=115
	X = X - 25		X=75	X=80
	write(x)		X=75	X=90
write(x)		X=115		

2. temporary update problem (dirty read)

- when one transaction updates an item and fails, but updated item is used by another transaction before item is changed or reverted to its last value.

Transaction 1 Transaction 2

Read (X)

$X = X - N$

write (X)

Read (X)

$X = X + M$

write (X)

→ T₁ fails and must
change the value of X
but T₂ has already
read the value.

read item (Y)

3. incorrect summary problem

- transaction is applying the aggregate function on some records while another transaction is updating these record. some may be calculated before or after the values have been updated.

T₁

T₂

sum = 0

read (A)

sum = sum + A

read (X)

$X = X - N$

write (X)

→ T₂ reads X after N is
subtracted and reads Y
before N is added.

read item (X)

sum = sum + X

read (Y)

sum = sum + Y

read (Y)

$Y = Y + N$

write (Y)

TYPES OF FAILURE

- transaction fails in the middle of the execution.

1. System crash.

a hardware, software or network error occurs in the computer system

hardware crashes are usually main memory failure.

2. Transaction or system error.

- operations such as integer overflow or division by zero.
- logical programming error or user itself could interrupt.

3. Local errors or exception condition.

- data for transaction cannot be found.
- insufficient funds in a banking database, can cause fund withdrawal to be cancelled.

4. Concurrency control enforcement.

- may abort a transaction because it violates serializability.
or to resolve a state of deadlock among several transaction.

— execution of multiple transactions in parallel does not produce any unexpected or incorrect results

5. Disk failure

- some disk block may lose their data because of read or write malfunction

6. Physical problems.

- power or air-conditioning failure, fire, theft, etc..

TRANSACTION LOG

- mechanisms used for recovering DB from failure.
- details about each step in the transaction so in case of failure, database can be recovered to the previous state.
- recovery requirement triggered by a rollback statement.
- programs abnormal termination.
- a system failure.

SYSTEM LOG

- append only file that is kept on disk so affected by only disk or catastrophic failure.
- main memory buffers (log) hold the last part of log file, so log entries are added to log main memory buffer.
- log file from disk is periodically backed up.
- T is a unique transaction ID generated automatically by the system.
- Log Records.
 1. [start_transaction, T] : transaction T has started execution.
 2. [write_item, T, x, old_value, new_value] : transaction T has changed value of database item x from old to new.
 3. [read_item, T, x] : transaction T has read value of database item x
 4. [commit, T] : transaction T has completed successfully and can be recorded permanently to the database.
 5. [abort, T] : transaction T has aborted.

CHECKPOINTS

- transaction logs eat up a lot of space.
- keeping track of every update and its maintenance may increase physical space of system.
- point of synchronization between database and transaction log is called checkpoint.
- checkpoint used to declare a point before which DBMS is in a consistent state.
- all buffers are force written to secondary storage.

SCHEDULE

- schedule S of n transaction $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions subject to the constraint that for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i .

Transaction T_1 : $r_1(x)$; $w_1(x)$; $r_1(y)$; $w_1(y)$; c_1

Transaction T_2 : $r_2(x)$; $w_2(x)$; c_2

Schedule S : $r_1(x)$; $r_2(x)$; $w_1(x)$; $r_1(y)$; $w_2(x)$; $w_1(y)$; c_1 ; c_2

CONFLICTS

- two operations conflict if they satisfy all three

1. belong to different transaction
2. access the same item
3. at least one is a write-item operation.

example: S : $r_1(x)$; $r_2(x)$; $w_1(x)$; $r_1(y)$; $w_2(x)$; $w_1(y)$



T_1	T_2
read(x)	read(x)
write(x)	
read(y)	
	write(x)
write(y)	

SERIALIZABILITY OF SCHEDULES

- It is a sequence of database action that does not violate serializability property.
- each transaction appears to execute atomically and is isolated from other transactions effects.
- must produce same results as if the transactions were executed one at a time in some order.

TYPE OF SCHEDULE

Serial: for every transaction T participating in the schedule, all the operations of T are executed consecutively.

→ transactions to be independent

- two schedule are equivalent if they lead to the same result.
or the order of any two conflicting operations is the same.
- two schedules are result equivalent if they produce the same final state of the database.

S_1	S_2	
read(x)	read(x)	for $x = 100 \rightarrow$ may produce same result by accident.
$x = x + 10$	$x = x * 1.1$	
write(x)	write(x)	

- two schedule are conflict equivalent if order of any two conflicting operations is the same in both schedules.

CONFLICT SERIALIZABLE

Schedule is conflict serializable if it is conflict equivalent to some serial schedules.

→ Non-conflicting.

- reads and writes from same transaction.
- reads from different transactions.
- reads and writes from different transaction on different data items.

→ conflicting

- reads and writes from different transaction on same data item.

SCHEDULE A

T1	T2
read(x)	
x = x - N	
write(x)	
read(y)	
y = y + N	
write(y)	
	read(x)
	x = x + N
	write(x)

SCHEDULE B

T1	T2
read(x)	
x = x - N	
write(x)	
	read(x)
	x = x + N
	write(x)
read(y)	
y = y + N	
write(y)	

∴ B is conflict equivalent to A ⇒ B is conflict serializable